

http://algs4.cs.princeton.edu

2.2 MERGESORT

- *mergesort*
- *bottom-up mergesort*
- *sorting complexity*
- *comparators*
- *stability*

Dy algoritme klasike për sortim: mergesort dhe quicksort

Critical components in the world's computational infrastructure.

- Studimi I plotë shkencor I veçorive të tyre ka mundësuar zhvillimin e tyre në system sorte praktike.
- Quicksort njihet si një ndër 10 algoritmet më të mira të shekullit të 20^{të} në shkencë dhe inxhinieri.

Mergesort. [kjo ligjëratë]



Quicksort. [ligjërata e ardhshme]





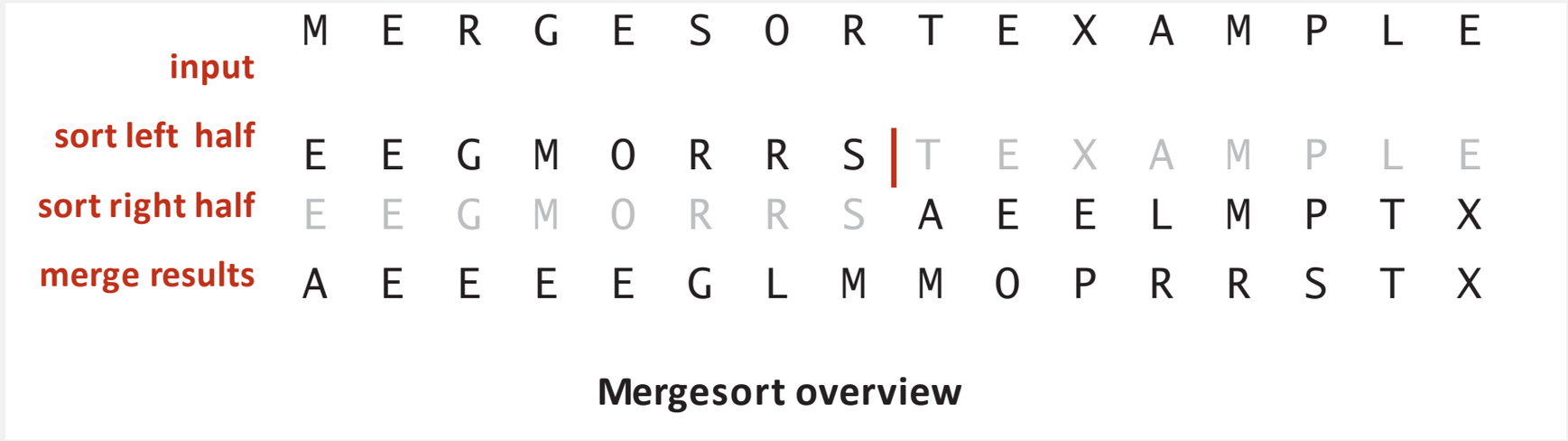
2.2 MERGESORT

- *mergesort*
- *bottom-up mergesort*
- *sorting complexity*
- *comparators*
- *stability*

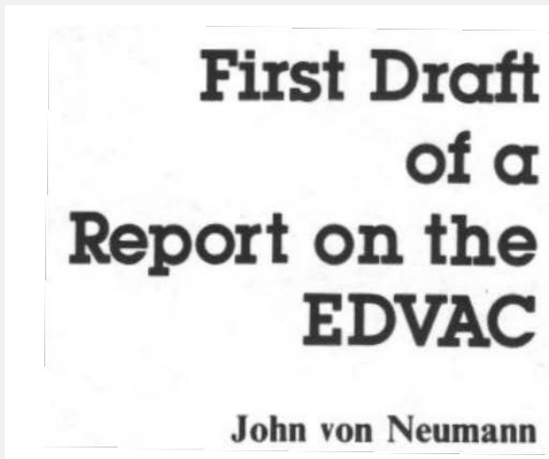
Mergesort

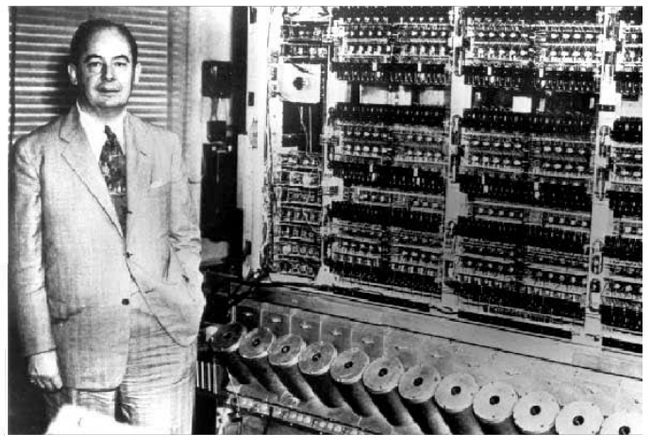
Plani bazë.

- Ndahet array në dy gjysma.
- Rekursivisht sortohet çdo gjysmë.
- Merge (bashkohen/ngjiten) gjysmat.



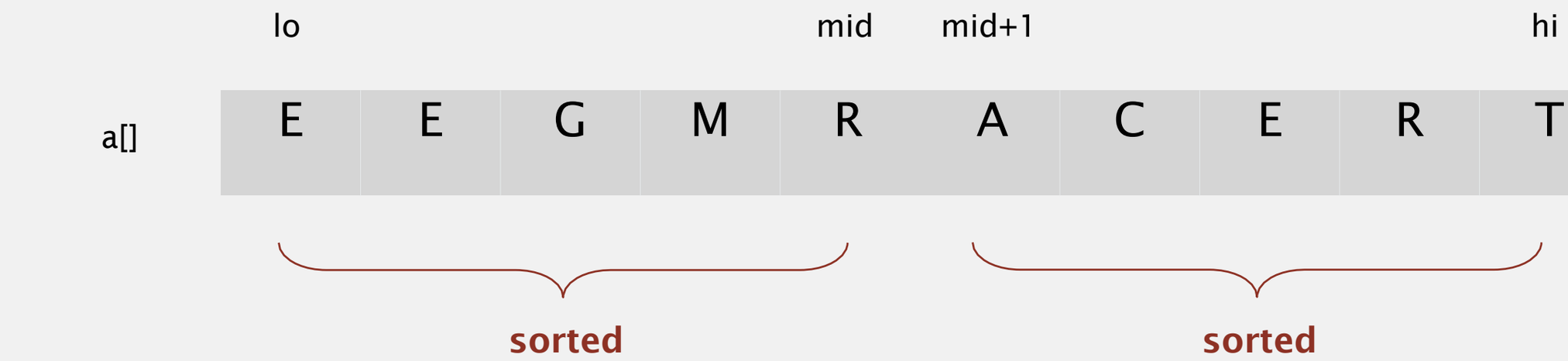
Mergesort overview





Merge demo

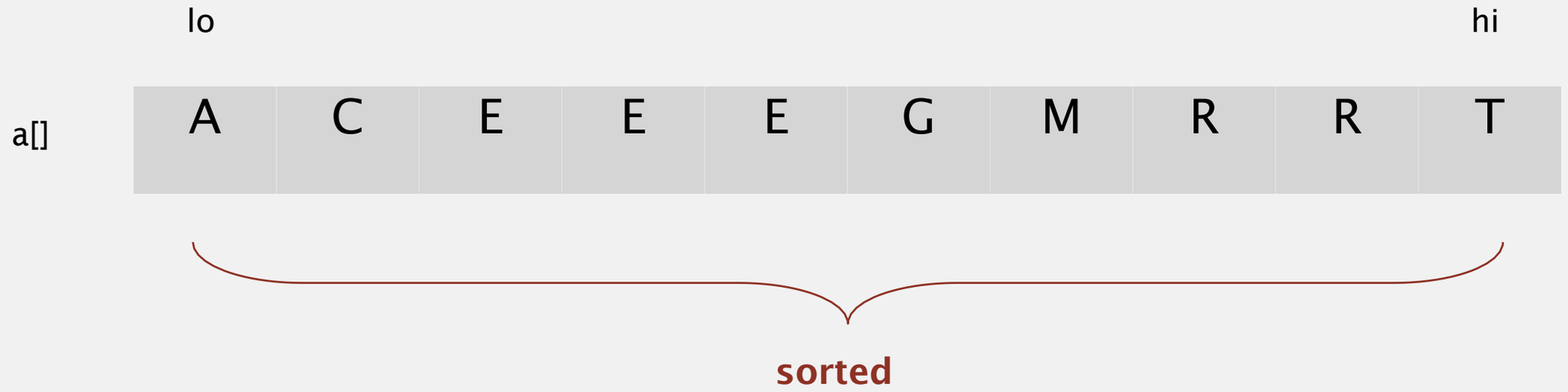
Qëllimi. Le të jenë dy subarrays të sortuara `a[lo]` deri `a[mid]` dhe `a[mid+1]` deri `a[hi]`, Zëvendoshet subarray të sortuar `a[lo]` to `a[hi]`.





Merge demo

Qëllimi. Le të jenë dy subarrays të sortuara `a[lo]` deri `a[mid]` dhe `a[mid+1]` deri `a[hi]`, Zëvendoshet subarray të sortuar `a[lo]` to `a[hi]`.



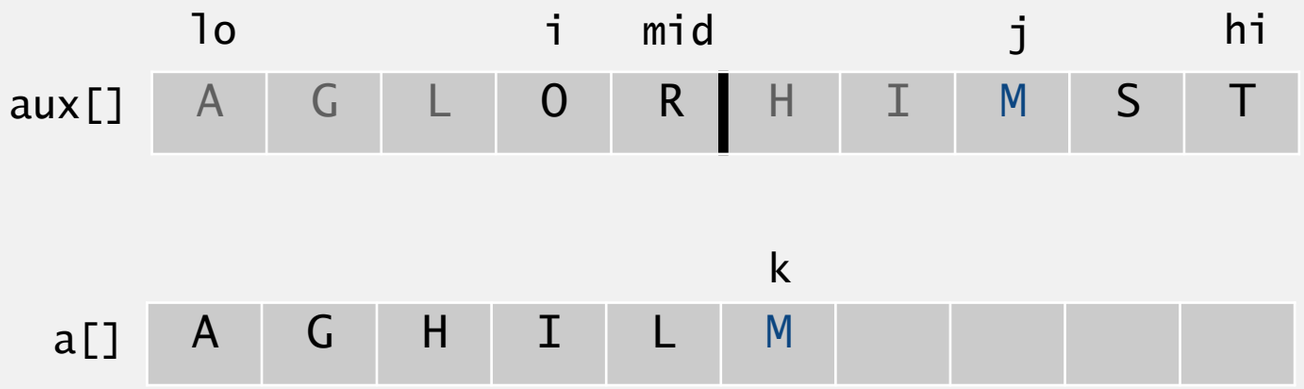
Merging: Java implementimi

```
private static void merge(Comparable[] a, Comparable[] aux, int lo, int mid, int hi)
{
   for (int k = lo; k <= hi; k++)
      aux[k] = a[k];

   int i = lo, j = mid+1;
   for (int k = lo; k <= hi; k++)
   {
      if      (i > mid)                   a[k] = aux[j++];
      else if (j > hi)                    a[k] = aux[i++];
      else if (less(aux[j], aux[i])) a[k] = aux[j++];
      else                                a[k] = aux[i++];
   }
}
```

copy

merge

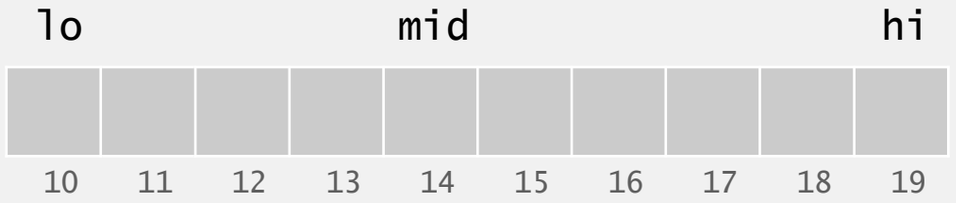


Mergesort: Java implementimi

```
public class Merge
{
   private static void merge(...)
   {  /* as before */  }

   private static void sort(Comparable[] a, Comparable[] aux, int lo, int hi)
   {
      if (hi <= lo) return;
      int mid = lo + (hi - lo) / 2;
      sort(a, aux, lo, mid);
      sort(a, aux, mid+1, hi);
      merge(a, aux, lo, mid, hi);
   }

   public static void sort(Comparable[] a)
   {
      Comparable[] aux = new Comparable[a.length];
      sort(a, aux, 0, a.length - 1);
   }
}
```



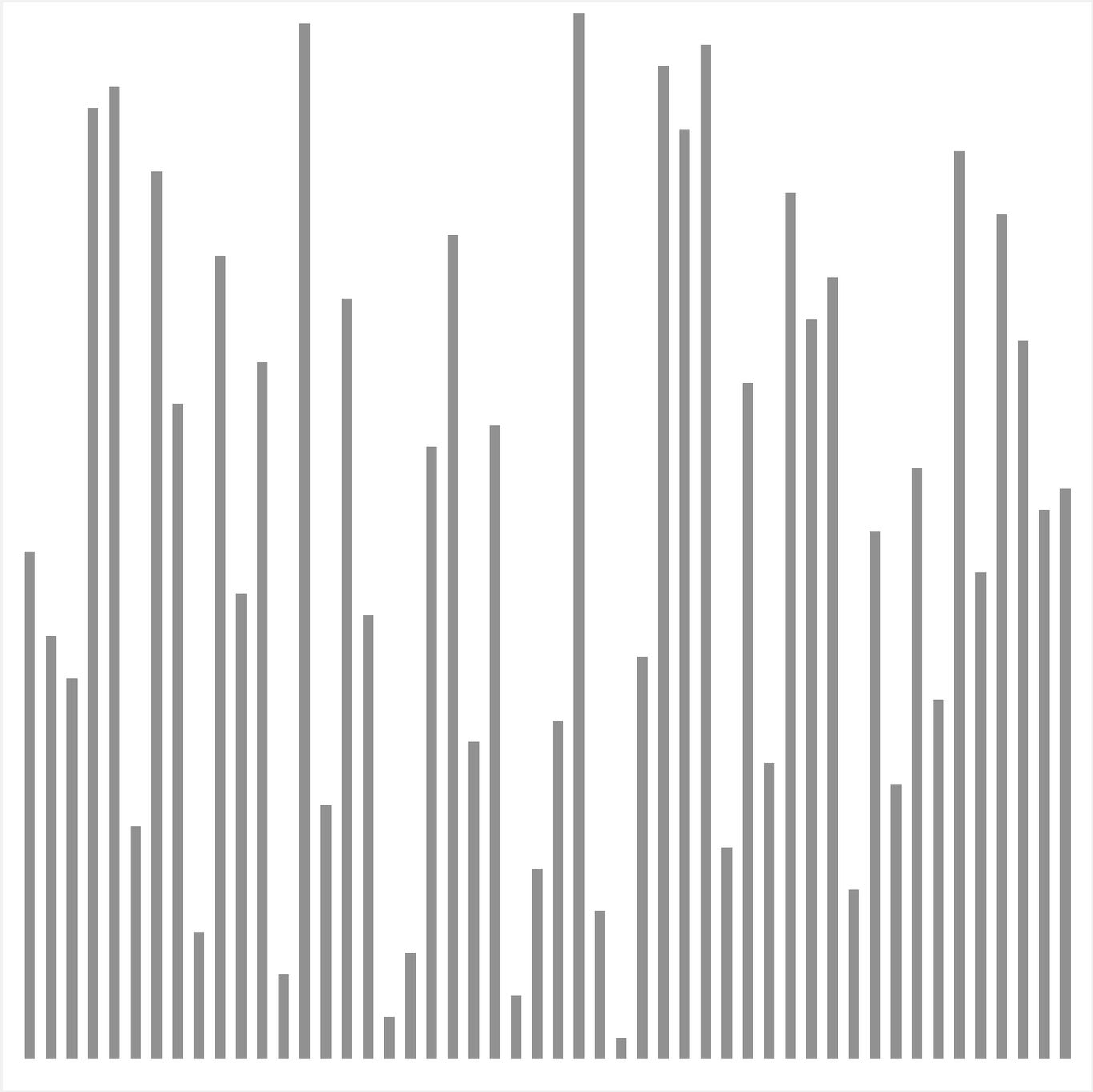
Mergesort: trace

	lo	hi	0	1	2	3	4	5	a[] 6	7	8	9	10	11	12	13	14	15
			M	E	R	G	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 0, 0, 1)	0	1	E	M	R	G	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 2, 2, 3)	2	3	E	M	G	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 0, 1, 3)	0	3	E	G	M	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 4, 4, 5)	4	5	E	G	M	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 6, 6, 7)	6	7	E	G	M	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 4, 5, 7)	4	7	E	G	M	R	E	O	R	S	T	E	X	A	M	P	L	E
merge(a, aux, 0, 3, 7)	0	7	E	E	G	M	O	R	R	S	T	E	X	A	M	P	L	E
merge(a, aux, 8, 8, 9)	8	9	E	E	G	M	O	R	R	S	E	T	X	A	M	P	L	E
merge(a, aux, 10, 10, 11)	10	11	E	E	G	M	O	R	R	S	E	T	A	X	M	P	L	E
merge(a, aux, 8, 9, 11)	8	11	E	E	G	M	O	R	R	S	A	E	T	X	M	P	L	E
merge(a, aux, 12, 12, 13)	12	13	E	E	G	M	O	R	R	S	A	E	T	X	M	P	L	E
merge(a, aux, 14, 14, 15)	14	15	E	E	G	M	O	R	R	S	A	E	T	X	M	P	E	L
merge(a, aux, 12, 13, 15)	12	15	E	E	G	M	O	R	R	S	A	E	T	X	E	L	M	P
merge(a, aux, 8, 11, 15)	8	15	E	E	G	M	O	R	R	S	A	E	E	L	M	P	T	X
merge(a, aux, 0, 7, 15)	0	15	A	E	E	E	E	G	L	M	M	O	P	R	R	S	T	X

result after recursive call

Mergesort: animation

50 random items



algorithm position

in order

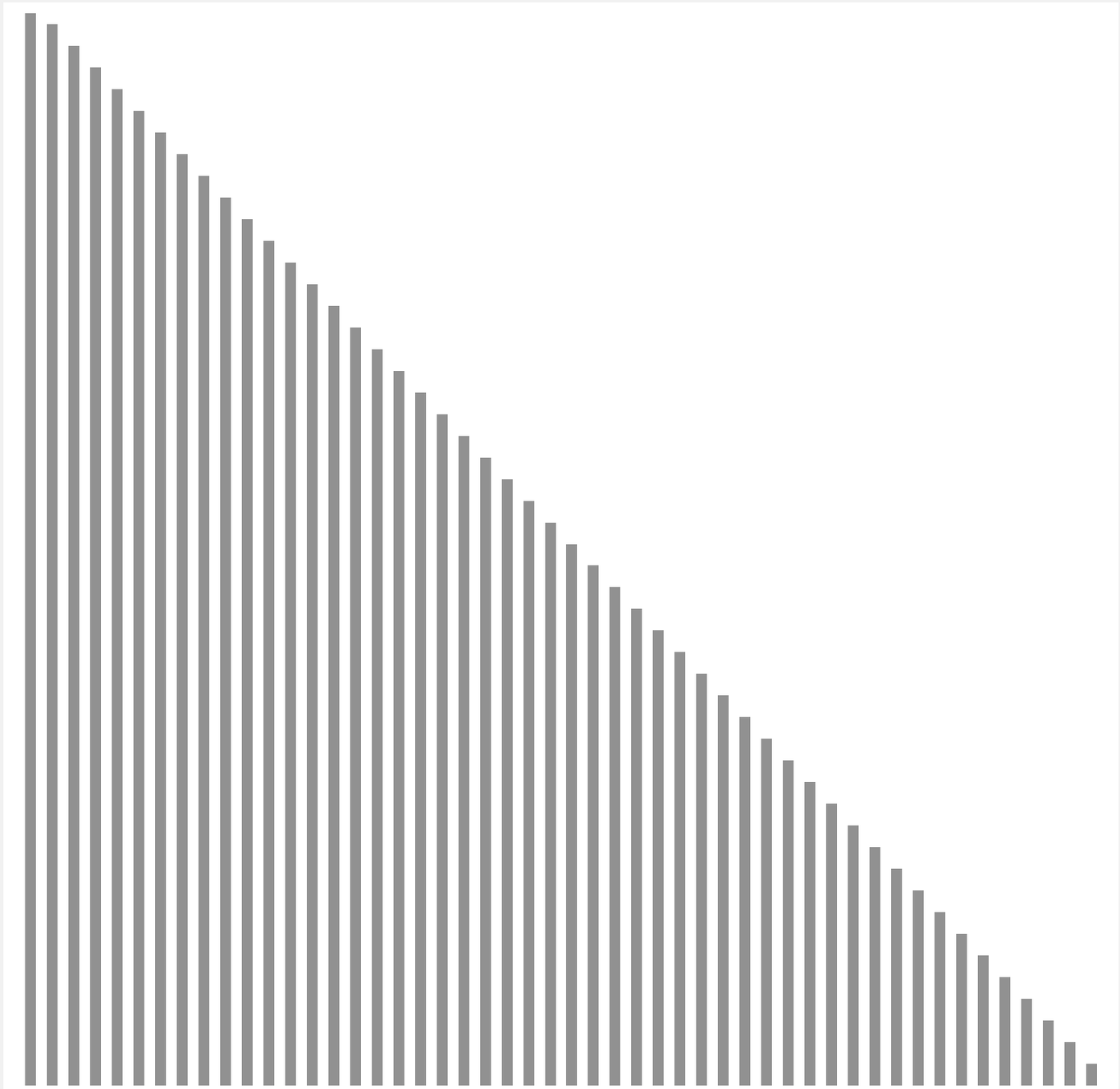
current subarray

not in order

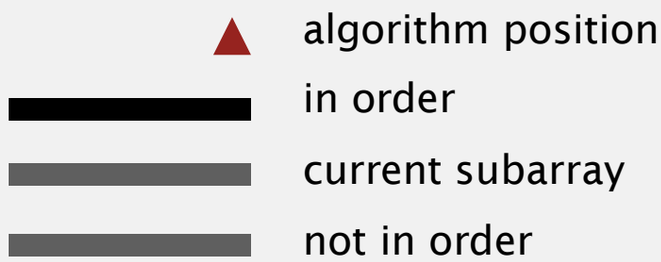
http://www.sorting-algorithms.com/merge-sort

Mergesort: animation

50 reverse-sorted items



http://www.sorting-algorithms.com/merge-sort



Mergesort: analiza empirike

Running time parashikimet:

- Laptop ekzekuton 10^8 krahasime/second.
- Superkompjuteri ekzekuton 10^{12} krahasime/second.

	insertion sort (N^2)			mergesort (N log N)		
computer	thousand	million	billion	thousand	million	billion
home	instant	2.8 hours	317 years	instant	1 second	18 min
super	instant	1 second	1 week	instant	instant	instant

Përfundim. Algorimet e mirë janë më mirë sesa superkompjuterët.

Mergesort: përmirësime praktike

Stop nëse veç e sortuar pjesa tjetër.

- A është elementi më i madh në gjysmën I $\leq$ elementi më i vogël në gjysmën II?
- Ndihmon kur array-at janë pjesërisht të rradhitura.

```
A  B  C  D  E  F  G  H  I  J   M  N  O  P  Q  R  S  T  U  V

A  B  C  D  E  F  G  H  I  J | M  N  O  P  Q  R  S  T  U  V
```

```
private static void sort(Comparable[] a, Comparable[] aux, int lo, int hi)
{
   if (hi <= lo) return;
   int mid = lo + (hi - lo) / 2;
   sort (a, aux, lo, mid);
   sort (a, aux, mid+1, hi);
   if (!less(a[mid+1], a[mid])) return;
   merge(a, aux, lo, mid, hi);
}
```

Java 6 system sort

Algoritmi bazë për sortimin e objekteve = mergesort.

- Cutoff to insertion sort.
- Stop-if-already-sorted test.
- Eliminate-the-copy-to-the-auxiliary-array trick.

Arrays.sort(a)



http://www.java2s.com/Open- S ource/Java/6.0- J DK-Modules/j2me/java/util/Arrays.java.html



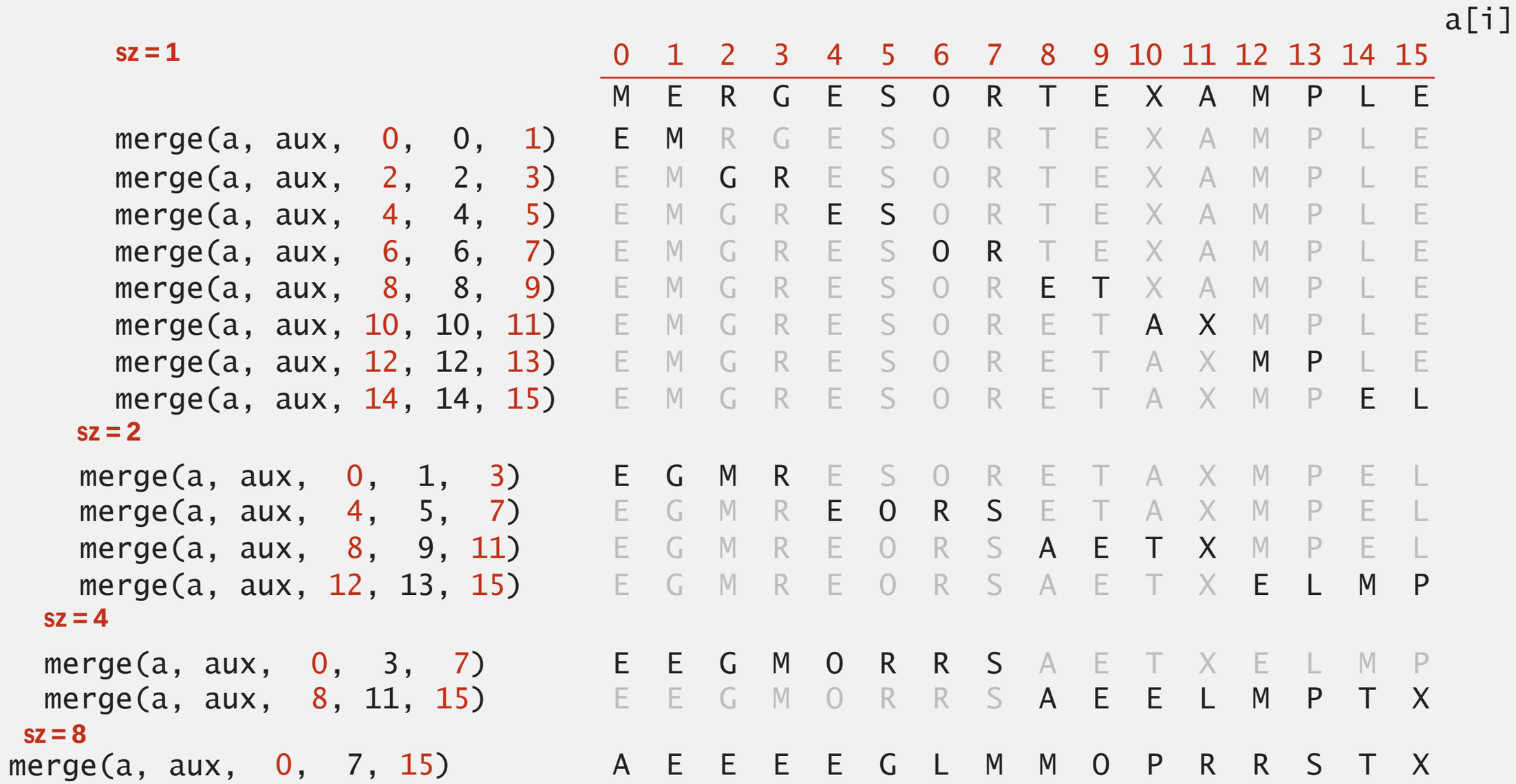
2.2 MERGESORT

- *mergesort*
- *bottom-up mergesort*
- *sorting complexity*
- *comparators*
- *stability*

Bottom-up mergesort

Plani bazë.

- Kalohet nëpër array, duke merge subarrays me madhësi 1.
- Përsëritet për subarrays të madhësisë 2, 4, 8, ...



Bottom-up mergesort: Java implementimi

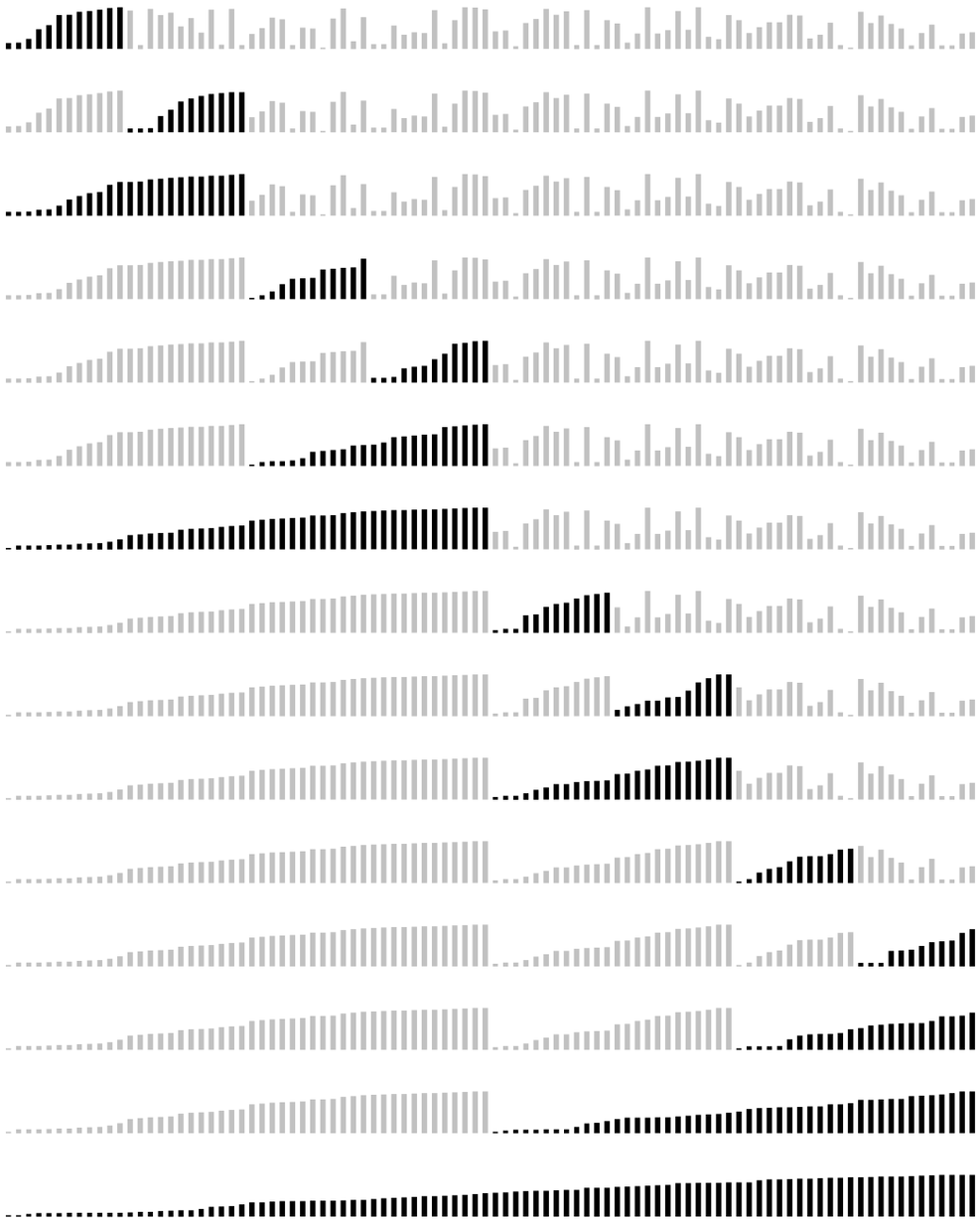
```
public class MergeBU
{
   private static void merge(...)
   {  /* as before */  }

   public static void sort(Comparable[] a)
   {
      int N = a.length;
      Comparable[] aux = new Comparable[N];
      for (int sz = 1; sz < N; sz = sz+sz)
         for (int lo = 0; lo < N-sz; lo += sz+sz)
            merge(a, aux, lo, lo+sz-1, Math.min(lo+sz+sz-1, N-1));
   }
}
```

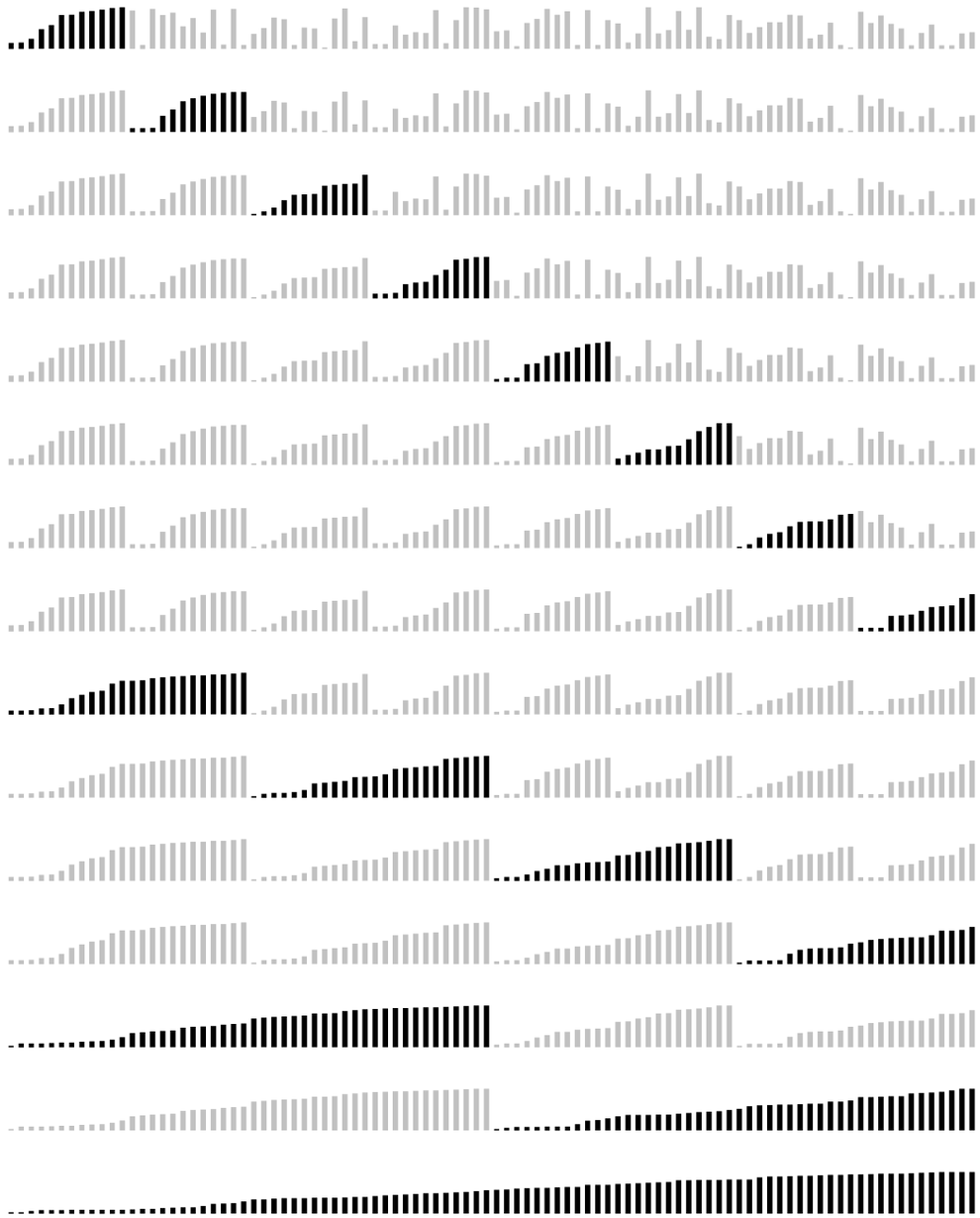
Mirëpo, 10% më i ngadaltë sesa recursive, top-down mergesort në sisteme tipike

Përfundim. Version i thjeshtë dhe jo-rekurziv i mergesort.

Mergesort: visualizim



top- down mergesort (cutoff = 12)



bottom-up mergesort (cutoff = 12)

Mergesort natyral

Idea. Shfrytëzohet rradhitja ekzistuese duke u identifikuar kalimet natyrale nëpër subarrays.

input

1	5	10	16	3	4	23	9	13	2	7	8	12	14

first run (kalimi I)

1	5	10	16	3	4	23	9	13	2	7	8	12	14

second run (kalimi II)

1	5	10	16	3	4	23	9	13	2	7	8	12	14

merge dy kalimet

1	3	4	5	10	16	23	9	13	2	7	8	12	14

Tradeoff. Më pak kalime vs. ekstra krahasime për kalim për të idetifikuar kalimet.

Timsort

- Mergesort natyral.
- Pëdorë binary insertion sort pë të bërë kalimet e para

(nëse ka nevojë).



Tim Peters

```
Intro
-----
This describes an adaptive, stable, natural mergesort, modestly called
timsort (hey, I earned it <wink>).  It has supernatural performance on many
kinds of partially ordered arrays (less than lg(N!) comparisons needed, and
as few as N-1), yet as fast as Python's previous highly tuned samplesort
hybrid on random arrays.

In a nutshell, the main routine marches over the array once, left to right,
alternately identifying the next run, then merging it into the previous
runs "intelligently".  Everything else is complication for speed, and some
hard-won measure of memory efficiency.
...
```

Rezultati. Koha lineare në shumë arrays me rradhitje ekzistuese.

Përdorimi. Python, Java 7, GNU Octave, Android, ….

The Zen of Python



http://www.python.org/dev/peps/pep- 0020/

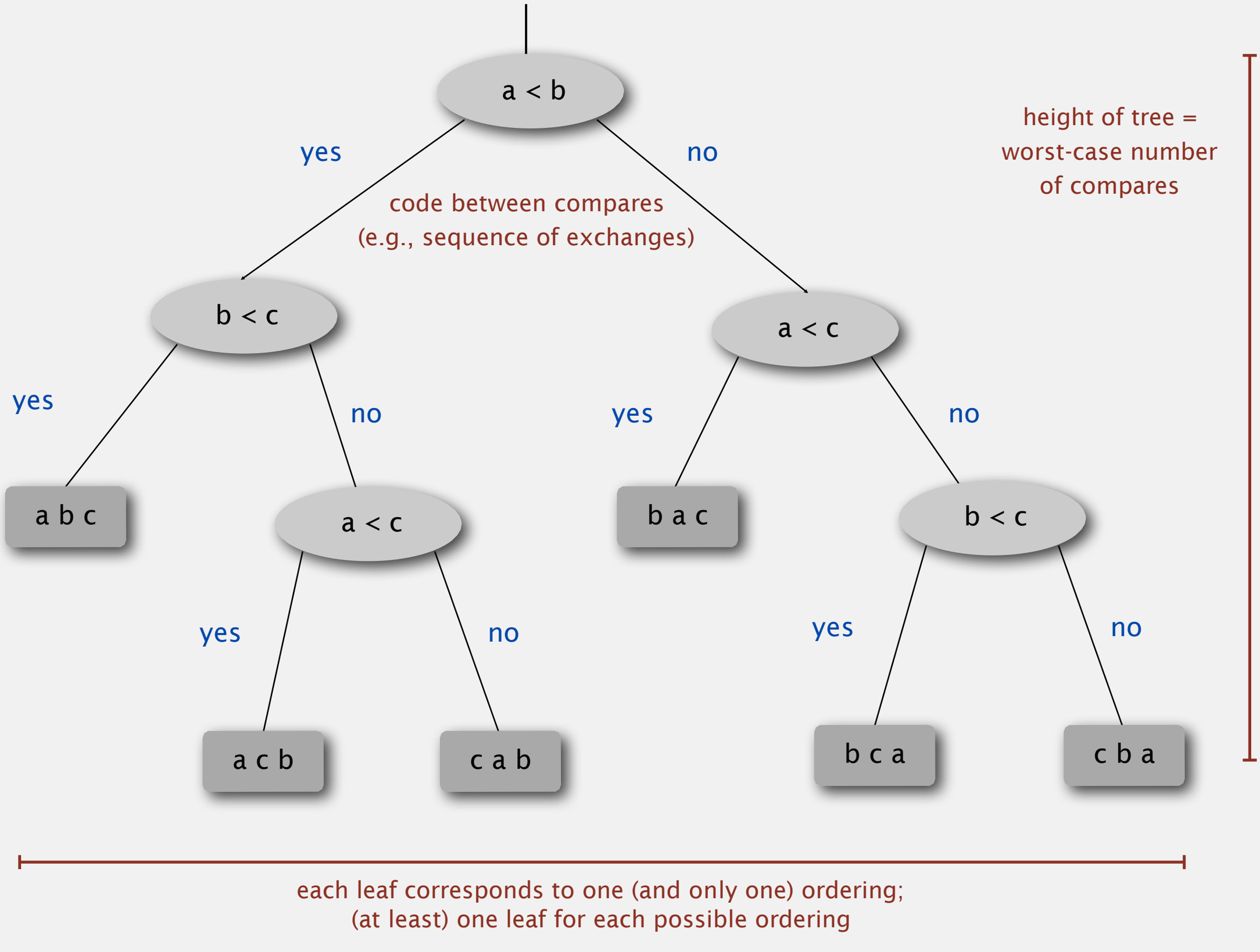
http://westmarch.sjsoft.com/2012/11/zen- of- python- poster/



2.2 MERGESORT

- *mergesort*
- *bottom-up mergesort*
- *sorting complexity*
- *comparators*
- *stability*

Decision tree (for 3 distinct keys a, b, and c)



Kompleksiciteti - rezultatet

Krahasimet? Mergesort është optimal për sa i përket numrit të krahasimeve.

Hapësira? Mergesort nuk është optimal për sa i përket hapësirës që shpenzon.





2.2 MERGESORT

- *mergesort*
- *bottom-up mergesort*
- *sorting complexity*
- *comparators*
- *stability*

Sort

NOC	Gold	Silver	Bronze	Total
United States (USA)	46	29	29	104
China (CHN)§	38	28	22	88
Great Britain (GBR)*	29	17	19	65
Russia (RUS)§	24	25	32	81
South Korea (KOR)	13	8	7	28
Germany (GER)	11	19	14	44
France (FRA)	11	11	12	34
Italy (ITA)	8	9	11	28
Hungary (HUN)§	8	4	6	18
Australia (AUS)	7	16	12	35

Sort

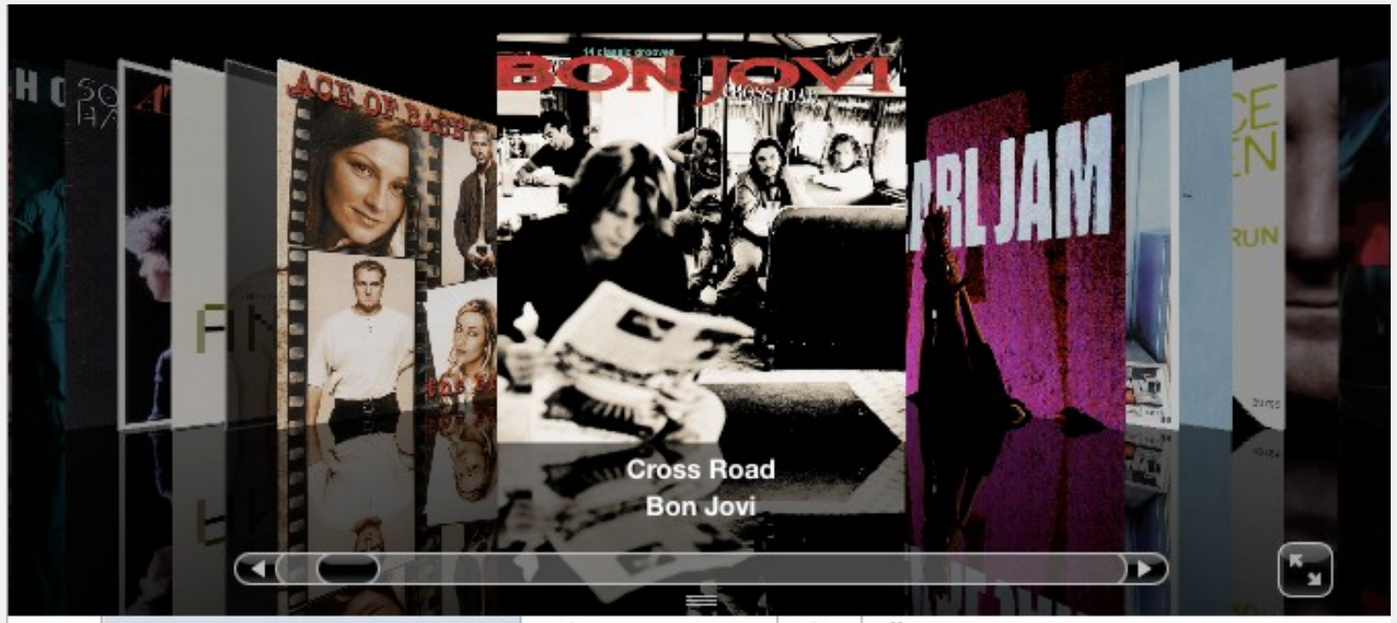
NOC	Gold	Silver	Bronze	Total
United States (USA)	46	29	29	104
China (CHN)§	38	28	22	88
Russia (RUS)§	24	25	32	81
Great Britain (GBR)*	29	17	19	65
Germany (GER)	11	19	14	44
Japan (JPN)	7	14	17	38
Australia (AUS)	7	16	12	35
France (FRA)	11	11	12	34
South Korea (KOR)	13	8	7	28
Italy (ITA)	8	9	11	28

Sort



	Name	Artist	Time	Album
12	Let It Be	The Beatles	4:03	Let It Be
13	Take My Breath Away	BERLIN	4:13	Top Gun - Soundtrack
14	Circle Of Friends	Better Than Ezra	3:27	Empire Records
15	Dancing With Myself	Billy Idol	4:43	Don't Stop
16	Rebel Yell	Billy Idol	4:49	Rebel Yell
17	Piano Man	Billy Joel	5:36	Greatest Hits Vol. 1
18	Pressure	Billy Joel	3:16	Greatest Hits, Vol. II (1978 - 1985) (Disc 2)
19	The Longest Time	Billy Joel	3:36	Greatest Hits, Vol. II (1978 - 1985) (Disc 2)
20	Atomic	Blondie	3:50	Atomic: The Very Best Of Blondie
21	Sunday Girl	Blondie	3:15	Atomic: The Very Best Of Blondie
22	Call Me	Blondie	3:33	Atomic: The Very Best Of Blondie
23	Dreaming	Blondie	3:06	Atomic: The Very Best Of Blondie
24	Hurricane	Bob Dylan	8:32	Desire
25	The Times They Are A-Changin'	Bob Dylan	3:17	Greatest Hits
26	Livin' On A Prayer	Bon Jovi	4:11	Cross Road
27	Beds Of Roses	Bon Jovi	6:35	Cross Road
28	Runaway	Bon Jovi	3:53	Cross Road
29	Rasputin (Extended Mix)	Boney M	5:50	Greatest Hits
30	Have You Ever Seen The Rain	Bonnie Tyler	4:10	Faster Than The Speed Of Night
31	Total Eclipse Of The Heart	Bonnie Tyler	7:02	Faster Than The Speed Of Night
32	Straight From The Heart	Bonnie Tyler	3:41	Faster Than The Speed Of Night
33	Holding Out For A Hero	Bonny Tyler	5:49	Meat Loaf And Friends
34	Dancing In The Dark	Bruce Springsteen	4:05	Born In The U.S.A.
35	Thunder Road	Bruce Springsteen	4:51	Born To Run
36	Born To Run	Bruce Springsteen	4:30	Born To Run
37	Jungleland	Bruce Springsteen	9:34	Born To Run

Sort



	Name	Artist	Time	Album
1	Alive	Pearl Jam	5:41	Ten
2	All Over The World	Pixies	5:27	Bossanova
3	All Through The Night	Cyndi Lauper	4:30	She's So Unusual
4	Allison Road	Gin Blossoms	3:19	New Miserable Experience
5	Ama, Ama, Ama Y Ensancha El ...	Extremoduro	2:34	Deltoya (1992)
6	And We Danced	Hooters	3:50	Nervous Night
7	As I Lay Me Down	Sophie B. Hawkins	4:09	Whaler
8	Atomic	Blondie	3:50	Atomic: The Very Best Of Blondie
9	Automatic Lover	Jay-Jay Johanson	4:19	Antenna
10	Baba O'Riley	The Who	5:01	Who's Better, Who's Best
11	Beautiful Life	Ace Of Base	3:40	The Bridge
12	Beds Of Roses	Bon Jovi	6:35	Cross Road
13	Black	Pearl Jam	5:44	Ten
14	Bleed American	Jimmy Eat World	3:04	Bleed American
15	Borderline	Madonna	4:00	The Immaculate Collection
16	Born To Run	Bruce Springsteen	4:30	Born To Run
17	Both Sides Of The Story	Phil Collins	6:43	Both Sides
18	Bouncing Around The Room	Phish	4:09	A Live One (Disc 1)
19	Boys Don't Cry	The Cure	2:35	Staring At The Sea: The Singles 1979-1985
20	Brat	Green Day	1:43	Insomniac
21	Breakdown	Deerheart	3:40	Deerheart
22	Bring Me To Life (Kevin Roen Mix)	Evanescence Vs. Pa...	9:48	
23	Californication	Red Hot Chili Pepp...	1:40	
24	Call Me	Blondie	3:33	Atomic: The Very Best Of Blondie
25	Can't Get You Out Of My Head	Kylie Minogue	3:50	Fever
26	Celebration	Kool & The Gang	3:45	Time Life Music Sounds Of The Seventies - C

Krahasimi – implementimi

Implementimi i krahasuesit:

- Definohet një (nested) class që implementon `Comparator` interface.
- Implementohet `compare()` metoda.

```
Arrays.sort(a, new Student.ByName());
```

Andrews	3	A	664-480-0023	097 Little
Battle	4	C	874-088-1212	121 Whitman
Chen	3	A	991-878-4944	308 Blair
Fox	3	A	884-232-5341	11 Dickinson
Furia	1	A	766-093-9873	101 Brown
Gazsi	4	B	766-093-9873	101 Brown
Kanaga	3	B	898-122-9643	22 Brown
Rohde	2	A	232-343-5555	343 Forbes

```
Arrays.sort(a, new Student.BySection());
```

Furia	1	A	766-093-9873	101 Brown
Rohde	2	A	232-343-5555	343 Forbes
Andrews	3	A	664-480-0023	097 Little
Chen	3	A	991-878-4944	308 Blair
Fox	3	A	884-232-5341	11 Dickinson
Kanaga	3	B	898-122-9643	22 Brown
Battle	4	C	874-088-1212	121 Whitman
Gazsi	4	B	766-093-9873	101 Brown



2.2 MERGESORT

- *mergesort*
- *bottom-up mergesort*
- *sorting complexity*
- *comparators*
- *stability*

Stabiliteti

A typical application. First, sort by name; then sort by section.

```
Selection.sort(a, new Student.ByName());
```

Andrews	3	A	664-480-0023	097 Little
Battle	4	C	874-088-1212	121 Whitman
Chen	3	A	991-878-4944	308 Blair
Fox	3	A	884-232-5341	11 Dickinson
Furia	1	A	766-093-9873	101 Brown
Gazsi	4	B	766-093-9873	101 Brown
Kanaga	3	B	898-122-9643	22 Brown
Rohde	2	A	232-343-5555	343 Forbes

```
Selection.sort(a, new Student.BySection());
```

Furia	1	A	766-093-9873	101 Brown
Rohde	2	A	232-343-5555	343 Forbes
	3	A	991-878-4944	308 Blair
	3	A	884-232-5341	11 Dickinson
	3	A	664-480-0023	097 Little
	3	B	898-122-9643	22 Brown
Gazsi	4	B	766-093-9873	101 Brown
Battle	4	C	874-088-1212	121 Whitman

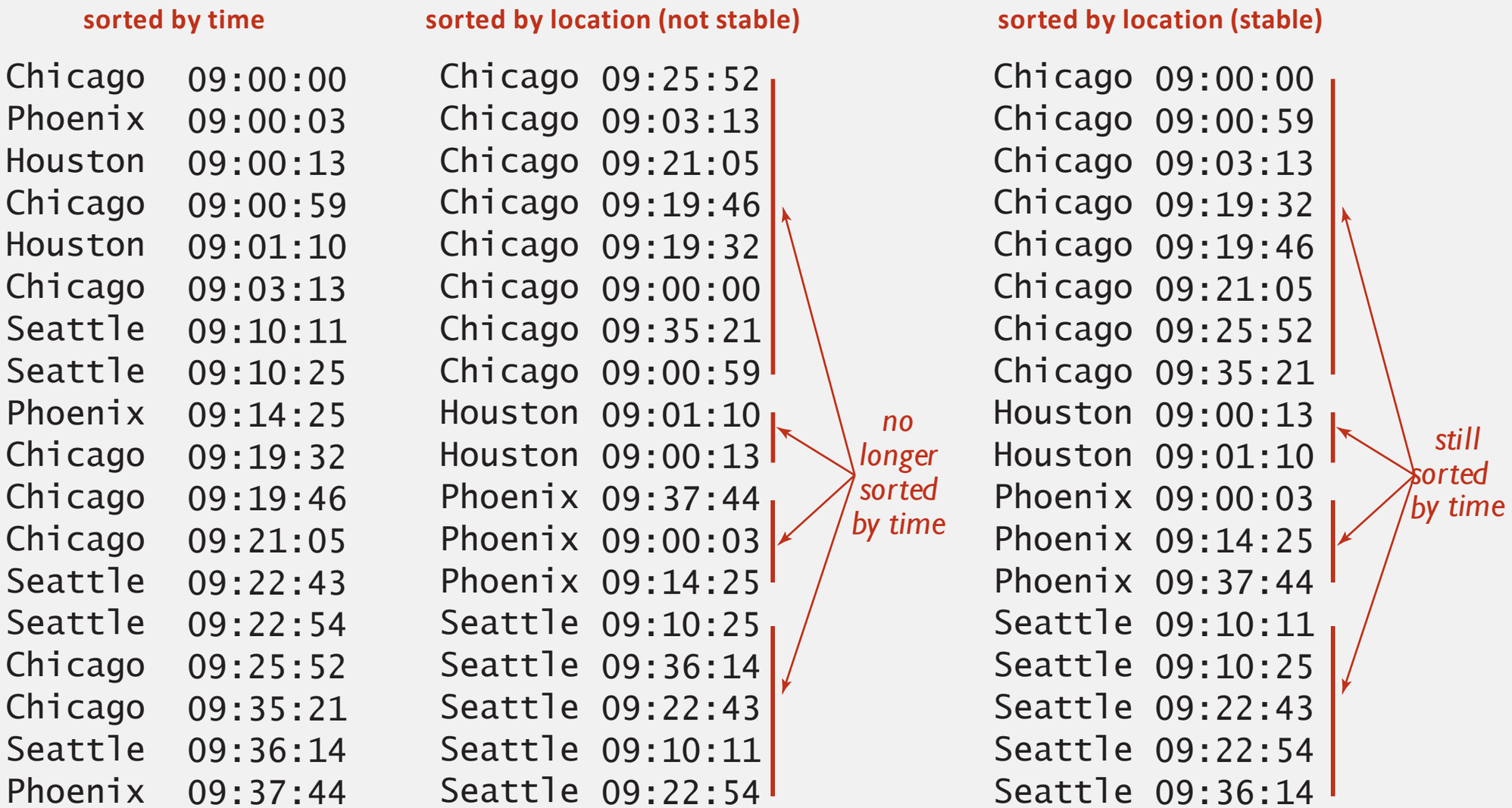
@#%&@! Studentët në sekcionin 3 nuk janë më të sortuar sipas emrit.

Një stable sort rradhitjen relative të elementeve me çelësa të barabartë.

Stabiliteti

Q. Cilët sorta janë stable?

A. Varet prej algoritmit (dhe implementimit).



Stabiliteti: insertion sort

Teoremë. Insertion sort është stable.

```
public class Insertion
{
   public static void sort(Comparable[] a)
   {
      int N = a.length;
      for (int i = 0; i < N; i++)
         for (int j = i; j > 0 && less(a[j], a[j-1]); j--)
            exch(a, j, j-1);
   }
}
```

i	j	0	1	2	3	4
0	0	B_1	A_1	A_2	A_3	B_2
1	0	A_1	B_1	A_2	A_3	B_2
2	1	A_1	A_2	B_1	A_3	B_2
3	2	A_1	A_2	A_3	B_1	B_2
4	4	A_1	A_2	A_3	B_1	B_2

Vërtetim. Elementet e barabarta kurrë nuk kalojnë njëri-tjetrin.

Stabiliteti: selection sort

Teoremë. Selection sort nuk është stable.

```
public class Selection
{
   public static void sort(Comparable[] a)
   {
      int N = a.length;
      for (int i = 0; i < N; i++)
      {
         int min = i;
         for (int j = i+1; j < N; j++)
            if (less(a[j], a[min]))
               min = j;
         exch(a, i, min);
      }
   }
}
```

i	min	0	1	2
0	2	B_1	B_2	A
1	1	A	B_2	B_1
2	2	A	B_2	B_1
		A	B_2	B_1

Vërtetim. Ndërrimet (e largëta) mund ta lejojnë që elementet e barabarta të tejkalojnë njëri-tjetrin

Stabiliteti: shellsort

Proposition. Shellsort sort is not stable.

```
public class Shell
{
   public static void sort(Comparable[] a)
   {
      int N = a.length;
      int h = 1;
      while (h < N/3) h = 3*h + 1;
      while (h >= 1)
      {
         for (int i = h; i < N; i++)
         {
            for (int j = i; j > h && less(a[j], a[j-h]); j -= h)
               exch(a, j, j-h);
         }
         h = h/3;
      }
   }
}
```

h	0	1	2	3	4
	B_1	B_2	B_3	B_4	A_1
4	A_1	B_2	B_3	B_4	B_1
1	A_1	B_2	B_3	B_4	B_1
	A_1	B_2	B_3	B_4	B_1

Vërtetim. Ndërrimet (e largëta) mund ta lejojnë që elementet e barabarta të tejkalojnë njëri-tjetrin

Stabiliteti: mergesort

Teoremë. Mergesort është stable.

```
public class Merge
{
   private static void merge(...)
   {  /* as before */  }

   private static void sort(Comparable[] a, Comparable[] aux, int lo, int hi)
   {
      if (hi <= lo) return;
      int mid = lo + (hi - lo) / 2;
      sort(a, aux, lo, mid);
      sort(a, aux, mid+1, hi);
      merge(a, aux, lo, mid, hi);
   }

   public static void sort(Comparable[] a)
   {  /* as before */  }
}
```

Vërtetim. Merr nga subarray e majtë nëse çelësat janë të barabartë.

Stabiliteti:mergesort

Teoremë. Mergesort është stable.

```
private static void merge(...)
{
   for (int k = lo; k <= hi; k++)
      aux[k] = a[k];

   int i = lo, j = mid+1;
   for (int k = lo; k <= hi; k++)
   {
      if      (i > mid)                  a[k] = aux[j++];
      else if (j > hi)                   a[k] = aux[i++];
      else if (less(aux[j], aux[i])) a[k] = aux[j++];
      else                               a[k] = aux[i++];
   }
}
```

0	1	2	3	4	5	6	7	8	9	10
A_1	A_2	A_3	B	D	A_4	A_5	C	E	F	G

Vërtetim. Merr nga subarray e majtë nëse çelësat janë të barabartë.

Sortimi - përmbledhje

	inplace?	stable?	best	average	worst	remarks
selection	✓		$\frac{1}{2} N^2$	$\frac{1}{2} N^2$	$\frac{1}{2} N^2$	N exchanges
insertion	✓	✓	N	$\frac{1}{4} N^2$	$\frac{1}{2} N^2$	use for small N or partially ordered
shell	✓		$N \log_3 N$	?	$c\, N^{3/2}$	tight code; subquadratic
merge		✓	$\frac{1}{2} N \lg N$	$N \lg N$	$N \lg N$	$N \log N$ guarantee; stable
timsort		✓	N	$N \lg N$	$N \lg N$	improves mergesort when preexisting order
?	✓	✓	N	$N \lg N$	$N \lg N$	holy sorting grail